

UNITÉ 6 · SEMAINE 9

Signaux et mémoire partagée

📅 Semaine 9 · dates à confirmer </> Laboratoire TD5 (corrigé + défi IA) ~ 3 animations

LABORATOIRE TD5 · CORRIGÉ

Signaux et mémoire partagée (`mmap` , `shm`)

Deux façons pour des processus de communiquer sans tubes: se prévenir par des signaux et partager des pages de mémoire. Ensemble, elles permettent à un processus de laisser un message en mémoire et de « sonner » chez l'autre.

✓ **Devoir : rapport du TP**

Rendez ce TP avec un court rapport en LaTeX (le strict nécessaire). Ce qu'il doit couvrir, le modèle, un exemple et le guide de Prism sont dans l'onglet Devoir. IA générative autorisée, avec déclaration.

EXERCICES

- | | | | |
|---|--|---|--|
| 1 | Capter SIGUSR1 avec <code>sigaction</code> | 4 | Partager du code: une « bibliothèque » faite à la main |
| 2 | Message par <code>mmap</code> + un signal | 5 | Mémoire partagée System V |
| 3 | Protections d'un <code>mmap</code> anonyme | | |

TD5 · 1

● Corrigé · testé

Capter SIGUSR1 avec `sigaction(2)`

Énoncé. `mysignal` affiche son pid, installe pour SIGUSR1 un gestionnaire qui affiche le numéro du signal, attend des signaux et rétablit le gestionnaire par défaut. Testez depuis un autre terminal avec `kill` : SIGUSR1, la séquence SIGSTOP → SIGUSR1 → SIGCONT, et SIGKILL.

Solution

```
C tds/td5/mysignal.c

/*
 * TD5 - exercise 1: catch SIGUSR1 with sigaction(2).
```

```

* ./mysignal [n]      handles SIGUSR1 n times (default 3), then restores
the default action
* From another terminal: kill -USR1 <pid>, kill -STOP <pid>, kill -CONT
<pid>, kill -KILL <pid>
* After the n-th SIGUSR1 the default action is back: the next SIGUSR1
terminates the process.
*/
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

static volatile sig_atomic_t received = 0;

static void on_usr1(int sig)
{
    /* Only async-signal-safe functions here: no printf (it may be
interrupted mid-buffer). */
    static const char msg[] = "handler: received signal ";
    char digits[4] = { (char)('0' + sig / 10), (char)('0' + sig % 10), '\n',
0 };
    if (write_all(STDERR_FILENO, msg, sizeof msg - 1) == -1 ||
write_all(STDERR_FILENO, digits, 3) == -1)
        return;
    received++;
}

int main(int argc, char *argv[])
{
    int max = argc > 1 ? atoi(argv[1]) : 3;
    struct sigaction sa = { 0 }, old;
    sigset_t block, wait_mask;

    printf("pid %d: waiting for SIGUSR1 (%d times)\n", getpid(), max);
    fflush(stdout);

    sa.sa_handler = on_usr1;
    sigemptyset(&sa.sa_mask);          /* nothing extra blocked while the
handler runs */
    sa.sa_flags = 0;
    exit_if(sigaction(SIGUSR1, &sa, &old) == -1, "sigaction");

    /* Block SIGUSR1 outside sigsuspend: a signal arriving between the test

```

```

of `received`
    and the wait stays pending instead of being lost (the pause() race).
*/
sigemptyset(&block);
sigaddset(&block, SIGUSR1);
exit_if(sigprocmask(SIG_BLOCK, &block, &wait_mask) == -1, "sigprocmask");
sigdelset(&wait_mask, SIGUSR1);

while (received < max)
    sigsuspend(&wait_mask);          /* atomically: unblock SIGUSR1 and
sleep */

exit_if(sigaction(SIGUSR1, &old, NULL) == -1, "sigaction restore");
printf("default action restored: the next SIGUSR1 terminates me\n");
fflush(stdout);
exit_if(sigprocmask(SIG_UNBLOCK, &block, NULL) == -1, "sigprocmask");
for (;;)
    pause();
}

```

Sortie réelle

```

$ ./mysignal 2 & P=$!; sleep 0.3; kill -USR1 $P; sleep 0.2; kill -STOP $P;
kill -USR1 $P; sleep 0.2; kill -CONT $P; sleep 0.2; kill -USR1 $P; wait $P;
echo "exit status: $? (128 + 10 = killed by SIGUSR1)"
pid 357338: waiting for SIGUSR1 (2 times)
handler: received signal 10
handler: received signal 10
default action restored: the next SIGUSR1 terminates me
bash: line 1: 357338 User defined signal 1      ./mysignal 2
exit status: 138 (128 + 10 = killed by SIGUSR1)

```

- **Avec le processus arrêté (SIGSTOP)**, SIGUSR1 n'est pas perdu: il reste *en attente* et le gestionnaire s'exécute au SIGCONT. SIGSTOP et SIGKILL ne peuvent être ni capturés ni bloqués: ce sont la garantie de l'administrateur.
- **Le gestionnaire n'utilise que des fonctions async-signal-safe** (`write`, pas `printf`): le signal peut arriver au milieu d'un `printf` du programme et rentrer dans son tampon.
- `sigprocmask` + `sigsuspend` au lieu de `pause`: si le signal arrive entre le test de `received` et `pause`, `pause` dormirait pour toujours. Avec le signal bloqué, il attend en suspens; `sigsuspend` le débloque et dort en une seule étape atomique.

Animation A6.1 · Délivrance des signaux →

TD5 · 2

● Corrigé · testé

Un message par `mmap` et un signal

Énoncé. `receiver` projette un fichier de 128 octets, écrit son pid au début et attend SIGUSR1; quand il arrive, il affiche le message. `sender` projette le même fichier, lit le pid, copie son entrée standard dans la zone partagée et envoie SIGUSR1. Que se passe-t-il si l'on écrit plus de 128 caractères ? Quelle est la taille réelle du segment ?

Solution

C tds/td5/receiver.c

```
/*
 * TD5 - exercise 2: receiver. Maps block.txt (shared), writes its pid at the
 * start of the
 * mapping, waits for SIGUSR1 and prints the message that the sender left
 * after the pid.
 * ./receiver &      then      echo "hello" | ./sender
 */
#include <fcntl.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/mman.h>
#include <unistd.h>

#include "utils.h"

#define FILENAME "block.txt"
#define SIZE 128

static void on_usr1(int sig) { (void)sig; } /* only interrupts sigsuspend
*/

int main(void)
{
```

```

struct sigaction sa = { 0 };
sigset_t usr1, old, wait_mask;

/* 1. Block SIGUSR1 BEFORE publishing the pid: a sender can only signal
us after that,
    and the signal will wait (pending) until sigsuspend. */
sigemptyset(&usr1);
sigaddset(&usr1, SIGUSR1);
exit_if(sigprocmask(SIG_BLOCK, &usr1, &old) == -1, "sigprocmask");
sa.sa_handler = on_usr1;
sigemptyset(&sa.sa_mask);
exit_if(sigaction(SIGUSR1, &sa, NULL) == -1, "sigaction");

/* 2. The file must be at least SIZE bytes: mapping past the end of a
file gives SIGBUS. */
int fd = open(FILENAME, O_RDWR | O_CREAT | O_TRUNC, 0644);
exit_if(fd == -1, "open " FILENAME);
exit_if(ftruncate(fd, SIZE) == -1, "ftruncate");
char *shared = mmap(NULL, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd,
0);
exit_if(shared == MAP_FAILED, "mmap");
exit_if(close(fd) == -1, "close");          /* the mapping stays
valid */

/* 3. Publish our pid. */
pid_t me = getpid();
memcpy(shared, &me, sizeof me);
printf("receiver %d: waiting for SIGUSR1\n", me);
fflush(stdout);

/* 4. Sleep with SIGUSR1 unblocked, atomically. */
wait_mask = old;
sigdelset(&wait_mask, SIGUSR1);
sigsuspend(&wait_mask);
exit_if(sigprocmask(SIG_SETMASK, &old, NULL) == -1, "sigprocmask"); /*
SETMASK, not BLOCK */

shared[SIZE - 1] = '\0';                    /* never trust the
other side */
printf("received: %s", shared + sizeof me);
exit_if(munmap(shared, SIZE) == -1, "munmap");
return EXIT_SUCCESS;
}

```

C tds/td5/sender.c

```
int main(void)
{
    int fd = open(FILENAME, O_RDWR);
    exit_if(fd == -1, "open " FILENAME);
    char *shared = mmap(NULL, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd,
0);
    exit_if(shared == MAP_FAILED, "mmap");
    exit_if(close(fd) == -1, "close");

    pid_t pid;
    memcpy(&pid, shared, sizeof pid);
    printf("sender: the receiver is %d\n", pid);

    /* Copy stdin into the message area, keeping one byte for '\0'. */
    size_t off = sizeof pid, max = SIZE - 1;
    ssize_t n = 1;
    while (off < max && (n = read(STDIN_FILENO, shared + off, max - off)) >
0)
        off += (size_t)n;
    exit_if(n == -1, "read");
    shared[off] = '\0';

    exit_if(msync(shared, SIZE, MS_SYNC) == -1, "msync"); /* optional: also
push to the file */
    exit_if(kill(pid, SIGUSR1) == -1, "kill");
    exit_if(munmap(shared, SIZE) == -1, "munmap");
    return EXIT_SUCCESS;
}
```

Sortie réelle

```
$ ./receiver & sleep 0.3; echo 'hello from the sender' | ./sender; wait
receiver 357391: waiting for SIGUSR1
sender: the receiver is 357391
received: hello from the sender
```

C tds/td5/mmap_limits.c

```
/*
 * TD5 - exercise 2, last question: how big is the shared segment of a 128-
byte file?
 * The kernel maps whole pages. We map TWO pages of a 128-byte file and write
byte by byte:
```

```

*   bytes 0..127      belong to the file
*   bytes 128..4095   exist (same page) but are never written back to the
file
*   byte 4096         is in a page that lies entirely past the end of the
file: SIGBUS
* The signal is caught and siglongjmp brings us back to report where it
happened.
*/
#include <fcntl.h>
#include <setjmp.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/mman.h>
#include <unistd.h>

#include "utils.h"

#define FILENAME "limits.txt"
#define FILESIZE 128

static sigjmp_buf env;
static volatile sig_atomic_t caught;
static char *volatile map;          /* volatile: must survive the
siglongjmp */
static volatile long offset;

static void on_fault(int sig)
{
    caught = sig;
    siglongjmp(env, 1);
}

int main(void)
{
    const long page = sysconf(_SC_PAGESIZE);
    int fd = open(FILENAME, O_RDWR | O_CREAT | O_TRUNC, 0644);
    exit_if(fd == -1, "open");
    exit_if(ftruncate(fd, FILESIZE) == -1, "ftruncate");
    map = mmap(NULL, 2 * page, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    exit_if(map == MAP_FAILED, "mmap");

    struct sigaction sa = { 0 };
    sa.sa_handler = on_fault;
    sigemptyset(&sa.sa_mask);

```

```

    exit_if(sigaction(SIGSEGV, &sa, NULL) == -1 || sigaction(SIGBUS, &sa,
NULL) == -1, "sigaction");

    if (sigsetjmp(env, 1) == 0)
        for (offset = 0; offset < 2 * page; offset++)
            map[offset] = 'x';

    printf("file size %d bytes, page size %ld bytes, mapping of %ld bytes\n",
FILESIZE, page, 2 * page);
    printf("wrote bytes 0..%ld; byte %ld raised %s\n", offset - 1, offset,
        caught ? signame(caught) : "nothing");

    exit_if(munmap(map, 2 * page) == -1, "munmap");
    off_t end = lseek(fd, 0, SEEK_END);          /* the file did not
grow */
    exit_if(end == -1, "lseek");
    printf("file size after the writes: %lld bytes\n", (long long)end);
    exit_if(close(fd) == -1, "close");
    return EXIT_SUCCESS;
}

```

Sortie réelle

```

$ ./mmap_limits
file size 128 bytes, page size 4096 bytes, mapping of 8192 bytes
wrote bytes 0..4095; byte 4096 raised SIGBUS
file size after the writes: 128 bytes

```

La taille réelle est d'une page (4096 octets). Le noyau projette des pages entières: les octets 128 à 4095 existent en mémoire mais n'appartiennent pas au fichier (ils ne sont jamais écrits sur le disque et le fichier garde 128 octets). Une page située entièrement au-delà de la fin du fichier déclenche **SIGBUS** quand on la touche. C'est pourquoi `receiver` appelle `ftruncate` : projeter un fichier vide et y écrire donne aussi SIGBUS.



Corrigée

La version héritée rétablissait le masque avec `SIG_BLOCK` (elle ne le rétablissait pas: elle y ajoutait), appelait `printf` dans le gestionnaire, et le message écrasait le pid publié. Désormais, le pid occupe les premiers octets et le message vient après.

TD5 · 3

● Corrigé · sorties réelles

Protections d'un `mmap` anonyme

Énoncé. `map_anonymous` demande un entier avec

`mmap(NULL, sizeof(int), PROT, MAP_PRIVATE | MAP_ANONYMOUS, -1, 0)`, affiche le pointeur, lit `*p`, écrit 42 et relit. Compilez-le avec `PROT_NONE`, `PROT_READ`, `PROT_WRITE`, `PROT_EXEC` et `PROT_EXEC|PROT_WRITE`, et expliquez chaque résultat avec `mmap(2)`.

Solution



C tds/td5/map_anonymous.c

```

/*
 * TD5 - exercise 3: protections of an anonymous mapping.
 * PROT is given at compile time; the Makefile builds map_none, map_read,
map_write, map_exec
 * and map_ew (PROT_EXEC|PROT_WRITE). Each program reads *p, writes 42 and
reads it again.
 * Read mmap(2): on x86-64, PROT_WRITE implies read access; PROT_NONE forbids
everything.
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/mman.h>
#include <unistd.h>

#include "utils.h"

#ifndef PROT
#define PROT (PROT_READ | PROT_WRITE)
#endif

/* Readable name of the protection flags, e.g. "PROT_EXEC|PROT_WRITE". */
static const char *prot_name(int prot)
{
    static char name[64];
    name[0] = '\0';
    if (prot == PROT_NONE)
        return "PROT_NONE";
    if (prot & PROT_READ) strcat(name, "|PROT_READ");
    if (prot & PROT_WRITE) strcat(name, "|PROT_WRITE");
    if (prot & PROT_EXEC) strcat(name, "|PROT_EXEC");
    return name + 1;
}

int main(void)
{
    int *p = mmap(NULL, sizeof(int), PROT, MAP_PRIVATE | MAP_ANONYMOUS, -1,
0);
    exit_if(p == MAP_FAILED, "mmap");

    fprintf(stderr, "PROT = %s\n", prot_name(PROT));
    fprintf(stderr, "pointer: %p\n", (void *)p);
    fprintf(stderr, "read:    *p = %d\n", *p);          /* anonymous memory

```

```
starts zeroed */
    *p = 42;
    fprintf(stderr, "write:  *p = %d\n", *p);

    exit_if(munmap(p, sizeof(int)) == -1, "munmap");
    return EXIT_SUCCESS;
}
```

Sortie réelle

```
$ ./map_none
PROT = PROT_NONE
pointer: 0x742c16995000
bash: line 1: 357416 Segmentation fault      (core dumped) ./map_none
[exit status 139]
```

Sortie réelle

```
$ ./map_read
PROT = PROT_READ
pointer: 0x7b6b0d393000
read:  *p = 0
bash: line 1: 357431 Segmentation fault      (core dumped) ./map_read
[exit status 139]
```

Sortie réelle

```
$ ./map_write
PROT = PROT_WRITE
pointer: 0x7c7095214000
read:  *p = 0
write: *p = 42
```

Sortie réelle

```
$ ./map_exec
PROT = PROT_EXEC
pointer: 0x75b3a49d8000
bash: line 1: 357495 Segmentation fault      (core dumped) ./map_exec
[exit status 139]
```

Sortie réelle

```
$ ./map_ew
PROT = PROT_WRITE|PROT_EXEC
pointer: 0x7872e8140000
read:    *p = 0
write:   *p = 42
```

La MMU applique les permissions de chaque page: `PROT_NONE` interdit même la lecture; `PROT_READ` permet de lire (la mémoire anonyme commence à zéro) mais pas d'écrire; sur x86-64, `PROT_WRITE` implique la lecture. `PROT_EXEC` seul est un cas intéressant: sur cette machine (avec des clés de protection mémoire), Linux crée de la mémoire *en exécution seule* et la lecture échoue; sur les processeurs sans cette fonction, le même test lit 0. C'est pourquoi `mmap(2)` dit que l'effet de chaque combinaison « dépend de l'architecture ». Chaque violation est un défaut de page que le noyau transforme en SIGSEGV (statut 139).

TD5 · 4

● Corrigé · testé

Partager du code: une « bibliothèque » faite à la main

Énoncé. Simulez une bibliothèque partagée: `load_add` copie le code machine de `add(int, int)` dans un fichier de 2048 octets projeté avec `MAP_SHARED`; `use_add` projette ce fichier avec `PROT_EXEC`, l'utilise comme pointeur de fonction et appelle `f(42, 12)`. Que se passe-t-il avec plusieurs `use_add` à la fois?

Solution



C tds/td5/load_add.c

```

int add(int a, int b)
{
    return a + b;
}

int main(void)
{
    int fd = open(LIBFILE, O_RDWR | O_CREAT | O_TRUNC, 0644);
    exit_if(fd == -1, "open " LIBFILE);
    /* An empty file cannot be mapped usefully (access past its end =
    SIGBUS): give it its size. */
    exit_if(ftruncate(fd, SIZE) == -1, "ftruncate");

    char *code = mmap(NULL, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    exit_if(code == MAP_FAILED, "mmap");
    exit_if(close(fd) == -1, "close");

    /* Copy the first bytes of add(). Converting a function pointer to void *
    is not ISO C,
       but POSIX requires it to work (dlsym relies on it). */
    memcpy(code, (const void *)add, 64);

    exit_if(munmap(code, SIZE) == -1, "munmap"); /* MAP_SHARED: the bytes
    are in the file */
    return EXIT_SUCCESS;
}

```

C tds/td5/use_add.c

```
int main(void)
{
    int fd = open(LIBFILE, O_RDONLY);
    exit_if(fd == -1, "open " LIBFILE);

    void *code = mmap(NULL, SIZE, PROT_READ | PROT_EXEC, MAP_SHARED, fd, 0);
    exit_if(code == MAP_FAILED, "mmap");
    exit_if(close(fd) == -1, "close");

    int (*f)(int, int) = (int (*)(int, int))code;
    printf("f(42, 12) = %d\n", f(42, 12));

    exit_if(munmap(code, SIZE) == -1, "munmap");
    return EXIT_SUCCESS;
}
```

Sortie réelle

```
$ ./load_add && ./use_add
f(42, 12) = 54
```

Cela marche parce que `add` n'utilise aucune adresse absolue : son code est indépendant de la position. Plusieurs `use_add` partagent la même page physique (celle du cache de pages du fichier), comme tous les programmes partagent les pages de `libc.so`. Un vrai chargeur (`ld.so`, `dlopen`) résout aussi les symboles et les relocalisations. La version héritée projetait le fichier sans lui donner de taille et finissait en SIGBUS.

TD5 · 5

● Corrigé · testé

Mémoire partagée System V

Énoncé. (1) Créez ou obtenez un segment de 1024 octets avec `ftok` + `shmget`, attachez-le avec `shmat`, incrémentez son premier entier, affichez-le et détachez-le avec `shmdt`. Lancez-le plusieurs fois : qu'observez-vous ? (2) Avec `fork` dans une boucle de 4 tours, 16 processus enregistrent leur pid dans le segment (nombre de pids + tableau de pids) et le premier processus affiche la liste. Recommencez avec 32 processus.

Solution



C tds/td5/shm_counter.c

```

/*
 * TD5 - exercise 5: a System V shared memory segment that survives the
 processes.
 * ./shm_counter increments the first int of the segment and
 prints it
 * ./shm_counter --remove destroys the segment (IPC_RMID)
 * Run it several times: 1, 2, 3... The segment lives in the kernel until it
 is removed or
 * the machine reboots (list them with `ipcs -m`, remove with `ipcrm`).
 * The key comes from ftok(file, id): same file + same id = same key on this
 machine.
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <unistd.h>

#include "utils.h"

#define KEYFILE "/etc/passwd" /* any file that exists on every Linux
 machine */
#define KEYID 'F'
#define SIZE 1024

int main(int argc, char *argv[])
{
    key_t key = ftok(KEYFILE, KEYID);
    exit_if(key == -1, "ftok");
    int id = shmget(key, SIZE, IPC_CREAT | 0644);
    exit_if(id == -1, "shmget");

    if (argc > 1 && strcmp(argv[1], "--remove") == 0) {
        exit_if(shmctl(id, IPC_RMID, NULL) == -1, "shmctl IPC_RMID");
        printf("segment %d removed\n", id);
        return EXIT_SUCCESS;
    }

    int *counter = shmat(id, NULL, 0);
    exit_if(counter == (void *)-1, "shmat");
    counter[0]++; /* a fresh segment is zero-
 filled */

```

```
printf("segment %d (key 0x%08x): counter = %d\n", id, (unsigned)key,
counter[0]);
exit_if(shmdt(counter) == -1, "shmdt");
return EXIT_SUCCESS;
}
```

Sortie réelle

```
$ ./shm_counter; ./shm_counter; ./shm_counter; ./shm_counter --remove
segment 65593 (key 0x4605476d): counter = 1
segment 65593 (key 0x4605476d): counter = 2
segment 65593 (key 0x4605476d): counter = 3
segment 65593 removed
```

C tds/td5/shm_fork.c

```
/*
 * TD5 - exercise 5 (end): 16 processes (4 fork() in a loop) register their
 * pid in a shared
 * segment: segment[0] = number of pids, then the pids. The original process
 * prints the list.
 *
 * "count++; table[count] = pid" is NOT atomic: two processes can read the
 * same count and
 * one registration is lost. Build with -DNO_ATOMIC (Makefile target
 * shm_race) to see it;
 * the default version reserves the slot with an atomic fetch-and-add (a
 * preview of unit 7).
 * The segment is IPC_PRIVATE: only this process and its descendants (which
 * inherit the
 * attachment through fork) can see it.
 */
#include <sched.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/wait.h>
#include <unistd.h>

#include "utils.h"

#define FORKS 4 /* 2^4 = 16 processes */
#define SIZE 4096
```

```
int main(int argc, char *argv[])
{
    int forks = argc > 1 ? atoi(argv[1]) : FORKS;
    int id = shmget(IPC_PRIVATE, SIZE, IPC_CREAT | 0600);
    exit_if(id == -1, "shmget");
    int *seg = shmat(id, NULL, 0);
    exit_if(seg == (void *)-1, "shmat");
    /* Mark for deletion now: it disappears when the last process detaches
    (even on a crash). */
    exit_if(shmctl(id, IPC_RMID, NULL) == -1, "shmctl");

    pid_t original = getpid();
    int children = 0;
    for (int i = 0; i < forks; i++) {
        pid_t pid = fork();
        exit_if(pid == -1, "fork");
        if (pid == 0)
            children = 0;          /* the child starts with no children
of its own */
        else
            children++;
    }

#ifdef NO_ATOMIC
    int slot = seg[0] + 1;          /* read ... */
    sched_yield();                 /* ... (widen the window) ... */
    seg[0] = slot;                 /* ... write: not atomic */
#else
    int slot = __atomic_add_fetch(&seg[0], 1, __ATOMIC_SEQ_CST); /* read-
modify-write in one step */
#endif
    if (slot < SIZE / (int)sizeof(int))
        seg[slot] = getpid();

    while (children-- > 0)
        exit_if(wait(NULL) == -1, "wait");

    if (getpid() == original) {
        printf("%d processes registered (expected %d):", seg[0], 1 << forks);
        for (int i = 1; i <= seg[0] && i < SIZE / (int)sizeof(int); i++)
            printf(" %d", seg[i]);
        printf("\n");
    }
    exit_if(shmdt(seg) == -1, "shmdt");
}
```

```
return EXIT_SUCCESS;
}
```

Sortie réelle

```
$ ./shm_fork
16 processes registered (expected 16): 357564 357566 357567 357572 357568
357571 357569 357565 357574 357573 357570 357576 357578 357575 357577 357579
```

Sortie réelle

```
$ for i in 1 2 3 4 5; do ./shm_race 5 | cut -d: -f1; done
19 processes registered (expected 32)
28 processes registered (expected 32)
25 processes registered (expected 32)
23 processes registered (expected 32)
27 processes registered (expected 32)
```

- **Le segment survit aux processus:** le compteur fait 1, 2, 3... parce que le segment vit dans le noyau jusqu'à ce que quelqu'un le supprime (`IPC_RMID`, `ipcrm`) ou que la machine redémarre. `ipcs -m` le montre. La version héritée le supprimait à la première exécution et le phénomène était invisible.
- **Avec 32 processus, des enregistrements se perdent** (entre 16 et 25 sur 32 dans les cinq exécutions réelles ci-dessus): `seg[0]++` est lire-ajouter-écrire, et deux processus peuvent lire la même valeur. C'est une *condition de course*, le sujet de l'unité 7. La solution utilise un incrément atomique du processeur (`__atomic_add_fetch`).
- `ftok` utilise un fichier qui existe sur tout Linux (`/etc/passwd`); l'énoncé d'origine utilisait `/etc/bashrc`, qui n'existe pas sous Ubuntu.

Liste de vérification

Défi avec l'IA générative · Remplacer ``pause`` par ``sigsuspend``... et prouver la course ^

Objectif. Écrivez avec l'IA un programme qui perd un signal avec ``pause()`` (la course classique), mesurez sa fréquence et montrez que ``sigprocmask`` + ``sigsuspend`` la supprime.

Prompt de départ. Utilisez-le dans l'outil de votre choix (Claude, ChatGPT, Gemini, Copilot, Codex...) avec la version *corrigée* du laboratoire comme contexte. Demandez d'abord un plan et relisez chaque étape avant de l'accepter.

Prompt

Je veux démontrer expérimentalement la course de `pause()`. Aide-moi à écrire : (1) un récepteur qui fait ``while (!arrived) pause();`` avec un gestionnaire qui met `arrived = 1` ; (2) un émetteur qui envoie SIGUSR1 juste après que le récepteur a publié son pid ; (3) un script qui répète l'expérience 1000 fois avec une limite de 1 s et compte combien de fois le récepteur dort pour toujours. Ensuite, écrivons la version avec `sigprocmask` + `sigsuspend` et comparons. Explique la fenêtre exacte dans laquelle le signal se perd.

Comment vérifier le résultat

- La version avec ``pause()`` se bloque au moins quelques fois (si elle ne se bloque jamais, élargissez la fenêtre avec un ``usleep`` avant ``pause``).
- La version avec ``sigsuspend`` ne se bloque jamais en 1000 exécutions.
- Vous savez montrer la ligne exacte entre le test et l'attente.

Déclarez votre usage de l'IA: outil, prompts pertinents, ce que vous avez vérifié vous-même et ce que vous avez corrigé. L'important est que vous puissiez expliquer et vérifier chaque décision.